



LeetCode 263: Ugly Number

1. Problem Title & Link

Title: LeetCode 263: Ugly Number

Link: <https://leetcode.com/problems/ugly-number/>

2. Problem Statement (Short Summary)

An **Ugly Number** is a positive number whose prime factors are limited to:

2, 3, and 5

Given an integer n , return:

- true if n is an ugly number
- false otherwise

Examples of Ugly Numbers

1

2

3

4

5

6

8

9

10

12

15

25

30

Why?

$$6 = 2 \times 3$$

$$8 = 2 \times 2 \times 2$$

$$15 = 3 \times 5$$

$$30 = 2 \times 3 \times 5$$

Only factors 2, 3, and 5 are used.

Examples of Non-Ugly Numbers

7

11

14



21

22

49

Why?

$$14 = 2 \times 7$$

$$21 = 3 \times 7$$

$$49 = 7 \times 7$$

Prime factor 7 appears.

3. Examples (Input → Output)

Example 1

Input:

$$n = 6$$

Output:

true

Explanation:

$$6 = 2 \times 3$$

Only factors 2 and 3.

Example 2

Input:

$$n = 1$$

Output:

true

Explanation:

1 is considered an Ugly Number.

Example 3

Input:

$$n = 14$$

Output:

false

Explanation:

$$14 = 2 \times 7$$

Contains factor 7.

Example 4



Input:

$n = 25$

Output:

true

Explanation:

$25 = 5 \times 5$

Only factor 5.

4. Constraints

$-2^{31} \leq n \leq 2^{31} - 1$

Important:

$n \leq 0 \rightarrow$ always false

Ugly numbers must be positive.

5. Core Concept (Pattern / Topic)

Mathematics / Prime Factorization

Secondary Topics:

- Division
- Number Theory

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Find all prime factors of n .

Check whether every prime factor belongs to:

$\{2, 3, 5\}$

Works but requires complete prime factorization.

Better Observation

If a number is ugly:

It can only contain factors:

2

3

5

So keep removing them.

Example:

$n = 30$



Remove 2:

$$30 / 2 = 15$$

Remove 3:

$$15 / 3 = 5$$

Remove 5:

$$5 / 5 = 1$$

Reached:

1

Therefore:

Ugly Number

Non-Ugly Example

$$n = 14$$

Remove 2:

$$14 / 2 = 7$$

Now:

7

Cannot divide by:

2

3

5

Remaining value:

7

Not equal to 1.

Therefore:

Not Ugly

Key Insight

Keep dividing by:

2

3

5

until no longer possible.

If the final value becomes:

1

then all factors were 2, 3, and 5.

Otherwise:



Some other prime factor exists.

7. Visual / Intuition Diagram (ASCII Diagram)

Example 1

30

$30 \div 2 = 15$

$15 \div 3 = 5$

$5 \div 5 = 1$

Reached 1

Ugly Number ✓

Example 2

14

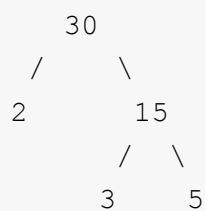
$14 \div 2 = 7$

Cannot divide by 2,3,5

Remaining = 7

Not Ugly ✗

Factor Tree



All leaves:

2,3,5

Ugly.

8. Pseudocode (Language Independent)

```

if n <= 0
    return false

while n % 2 == 0
  
```



```
n /= 2

while n % 3 == 0:
    n /= 3

while n % 5 == 0:
    n /= 5

return n == 1
```

9. Code Implementation

Python

```
class Solution:
    def isUgly(self, n: int) -> bool:

        if n <= 0:
            return False

        while n % 2 == 0:
            n //= 2

        while n % 3 == 0:
            n //= 3

        while n % 5 == 0:
            n //= 5

        return n == 1
```

Java

```
class Solution {

    public boolean isUgly(int n) {

        if(n <= 0)
            return false;

        while(n % 2 == 0)
            n /= 2;

        while(n % 3 == 0)
```



```
        n /= 3;

        while(n % 5 == 0)
            n /= 5;

        return n == 1;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(\log n)$

Reason:

Each division significantly reduces the value.

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting:

$n \leq 0$

Example:

$n = -6$

Expected:

false

Mistake 2

Checking only once.

Wrong:

if $n \% 2 == 0$:

$n //= 2$

Need:

while $n \% 2 == 0$:

$n //= 2$

because:

$8 = 2 \times 2 \times 2$



must remove all factors.

Mistake 3

Returning:

`n == 0`

instead of:

`n == 1`

Edge Cases

Input	Output
1	TRUE
2	TRUE
3	TRUE
5	TRUE
6	TRUE
14	FALSE
0	FALSE
-5	FALSE

12. Detailed Dry Run (Step-by-Step Table)

Input:

`n = 60`

Initial:

60

Operation	n
$\div 2$	30
$\div 2$	15
$\div 3$	5
$\div 5$	1

Final:

1

Return:

true

Another Dry Run

Input:



$n = 14$

Operation	n
$\div 2$	7
Stop	7

Final:

7

Return:

false

13. Common Use Cases (Real-Life / Interview)

Signal Processing

Ugly numbers are used in generating optimized sample sizes.

Dynamic Programming Problems

Foundation for:

Ugly Number II

Super Ugly Number

Number Theory

Prime factorization analysis.

Interview Purpose

Tests:

Division

Prime factors

Loop control

Mathematical reasoning

14. Common Traps (Important!)

Trap 1

Treating:

1

as non-ugly.

Actually:

1 $\rightarrow$ Ugly Number

by definition.

**Trap 2**

Not removing factors repeatedly.

Need:

while

not:

if

Trap 3

Trying to generate all factors.

Unnecessary.

Simply remove:

2,3,5

and check the remainder.

15. Builds To (Related LeetCode Problems)

Progression:

LC 263 - Ugly Number

↓

LC 264 - Ugly Number II

↓

LC 313 - Super Ugly Number

↓

LC 204 - Count Primes

↓

LC 2521 - Distinct Prime Factors

These strengthen Number Theory skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Prime Factorization	$O(\sqrt{n})$	$O(1)$	Fair
Remove 2,3,5 Factors	$O(\log n)$	$O(1)$	Best

Approach 1: Prime Factorization

Find every prime factor.

Check if all belong to:

{2,3,5}



More work than needed.

Approach 2: Repeated Division (Chosen)

Remove:

2

3

5

until impossible.

Check:

$n == 1$

Simple and optimal.

17. Why This Solution Works (Short Intuition)

An ugly number can only contain prime factors:

2,3,5

If we repeatedly remove all occurrences of these factors and eventually reach:

1

then no other prime factors existed.

If a value greater than 1 remains, some other prime factor must be present.

18. Variations / Follow-Up Questions

Follow-Up 1

Find the nth Ugly Number.

Example:

$n = 10$

Output:

12

(10th ugly number)

Related to **LeetCode 264**.

Follow-Up 2

Allow factors:

2,3,5,7

How would the solution change?

Follow-Up 3

Count ugly numbers within a range.



Example:

1 → 1000

Follow-Up 4

Find the next ugly number after a given value.

Example:

Input: 14

Output: 15

Interview Takeaway

The key observation is:

Ugly Number

=

Prime Factors Only From {2,3,5}

Therefore:

Keep dividing by 2, 3, and 5

and finally check:

$n == 1$

This transforms a prime-factorization problem into a very clean and efficient solution.